MANUAL TÉCNICO DEL PROGRAMADOR Y ARQUITECTURA DE SOFTWARE

Proyecto: Motor de Videojuego "Juego de Naves" en Consola mediante Clases

Institución: E.E.S.T. N° 1 "Eduardo Ader" Vicente López

Asignatura: Laboratorio de Programación (LPR) - 5° Año 3° División

Docente Evaluador: Mansilla Muñoz York Elías

Autora: Sofia Mojica (Desarrollo Individual)

Fecha: 15 de octubre de 2026

Versión: v1.0.0

CONTROL DE VERSIONES Y CHANGELOG

- Versión v1.0.0 (15/10/2026): Migración integral del paradigma estructurado previo hacia la Programación Orientada a Objetos mediante la encapsulación en clases independientes (Nave, Asteroide, Bala) y el control de variables de estado por métodos públicos.

---

1. ARQUITECTURA DEL SISTEMA Y ENTORNO DE DESARROLLO

- **Requisitos del Entorno:** Compilador g++ (MinGW-w64 GCC v16.2.0 o superior) con soporte para el estándar C++11 habilitado de manera global en la variable de entorno PATH, IDE Visual Studio Code con el paquete C/C++ Extension Pack activo, y shell de comandos Windows PowerShell 5.1+.
- **Estructura de Directorios del Repositorio:** Carpeta raíz juegodenaves/ organizada mediante exclusiones .gitignore, licencia MIT escolar, portada de presentación (README.md), carpeta de evidencias (capturas/combate_consola.png), directorio fuente (src/main.cpp) y el segmento de documentación académica formal (docs/ y docs/manuales/).

---

2. DOCUMENTACIÓN DE CLASES Y MÉTODOS DE BAJO NIVEL

A. Clase Nave

Encapsula el estado lógico del jugador utilizando modificadores de acceso restrictivos.

- **Sección Privada (private):** Resguarda las coordenadas físicas de posición int x, int y, junto al estado de integridad int vidas. Esto previene mutaciones o alteraciones accidentales desde la función global main(), garantizando el aislamiento de datos.
- **Sección Pública (public):** Expone el Constructor Nave(int _x, int _y, int _vidas) para la instanciación e inicialización en la memoria Stack, métodos de acceso seguro (getX, getY, getVidas), la función reducirVida() y los módulos procedimentales pintar(), borrar() y mover(char tecla).

B. Clases Asteroide y Bala

Modelan de forma independiente las entidades autónomas del sistema. Poseen sus propios constructores encapsulados y ejecutan subprogramas de actualización de posición (mover) que interactúan con las coordenadas de la pantalla mediante llamadas físicas a la función nativa gotoxy(), la cual invoca los controladores de la API de Windows mediante SetConsoleCursorPosition.

---

3. INSTRUCCIONES DE COMPILACIÓN Y DIAGNÓSTICO

- **Compilación de la Suite:** Desde la raíz del directorio del proyecto, ejecute en PowerShell:
  `g++ src/main.cpp -o src/juegodenaves.exe`
- **Lanzamiento del Binario:** Inicialice el Game Loop ejecutando la instrucción:
  `.\src\juegodenaves.exe`
- **Diagnóstico de Errores Frecuentes:** Si los elementos autónomos imprimen estelas en la pantalla, valide la exacta simetría de las coordenadas de texto invocadas en los métodos borrar() de cada clase. En caso de fallas de posicionamiento, asegúrese de que el IDE cuente con los privilegios de administrador del sistema requeridos para interactuar con las llamadas de la API GetStdHandle.

---

4. REFERENCIAS

- Kernighan, B. W., & Ritchie, D. M. (1988). *El lenguaje de programación C* (2.ª ed.). Prentice Hall.
- Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.